

Logistic Regression — Practice Problems

Instructor: Dr. Akash Gupta

Table of contents

Part 1: Questions	2
Part 2: Answers and Explanations	24

Part 1: Questions

Question 1

A Capital One team needs to predict whether a credit-card application will be approved (yes/no). Which model family is the natural starting point?

- a) Linear regression on the 0/1 outcome
- b) PCA
- c) Logistic regression — built for binary classification, outputs a probability between 0 and 1
- d) K-Means
- e) Polynomial regression

[☐ Jump to solution](#)

Question 2

Why is plain linear regression a poor choice for binary classification?

- a) Linear regression cannot be trained on any binary problem
- b) Predictions can fall outside $[0, 1]$ (you might get 1.4 or -0.2 for a “probability”); the model treats unit changes as equally important regardless of where on the 0-to-1 scale you start; and the squared-error loss assumes continuous outcomes
- c) Linear regression always produces probabilities greater than 1
- d) Linear regression requires more data than logistic regression
- e) Scikit-learn does not support linear regression on binary data

[☐ Jump to solution](#)

Question 3

A Lending Club team is modeling whether a borrower will default. Which of the following is the BEST framing for logistic regression?

- a) Predict the actual loss in dollars given default
- b) Cluster borrowers into segments
- c) Predict which credit card the borrower will apply for next
- d) Predict the probability that the borrower defaults — and convert to a yes/no decision via a threshold the team chooses
- e) Forecast next month’s loan demand

[☐ Jump to solution](#)

Question 4

True or False: logistic regression is a LINEAR classifier.

- a) True — its decision boundary in the original feature space is a hyperplane (a straight line in 2D, a flat plane in 3D, etc.)
- b) False — it produces an S-curve so its boundary is curved
- c) False — it uses tree splits internally
- d) True only for regression problems
- e) False — logistic regression is a tree model

[☐ Jump to solution](#)

Question 5

When would you choose Logistic Regression over a Random Forest for a fraud-detection problem?

- a) Always — Logistic Regression is the more accurate model
- b) When you need transparent, well-calibrated probabilities and human-readable coefficient interpretations (regulators, banking, healthcare); when the relationship is roughly linear in the features; or when you want a fast baseline before reaching for more complex models
- c) Never — Random Forest dominates in every scenario
- d) When the dataset is extremely small (under 30 rows)
- e) When there are no numeric features

[☐ Jump to solution](#)

Question 6

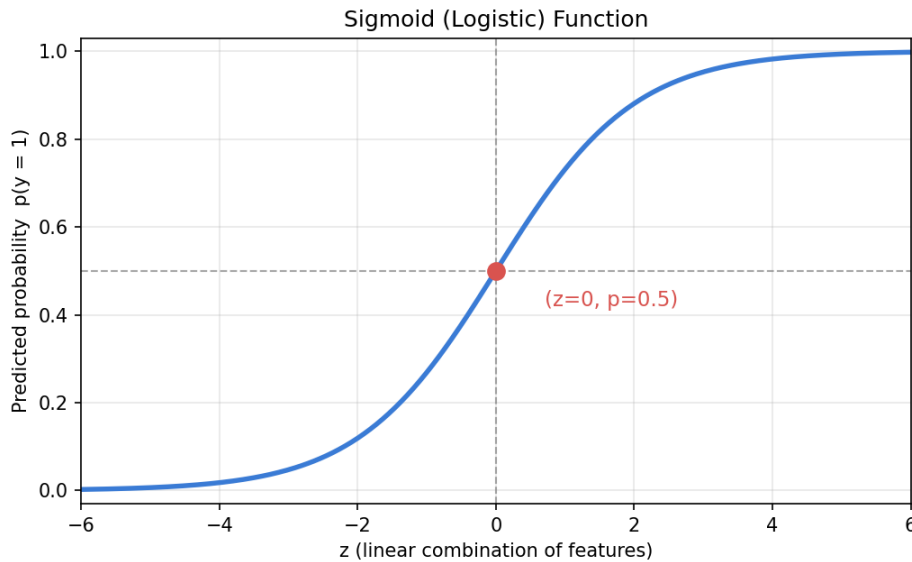
What does the **sigmoid (logistic) function** do?

- a) It computes the dot product of two vectors
- b) It returns the maximum of two numbers
- c) It exponentiates a vector and takes the sum
- d) It takes any real number z (positive, negative, or zero) and returns a value strictly between 0 and 1, with $z = 0$ mapped to 0.5 — exactly the shape needed to interpret the output as a probability
- e) It rounds to the nearest integer

[☐ Jump to solution](#)

Question 7

The plot below shows the sigmoid function used by logistic regression.



What is the predicted probability when the linear combination z equals 0?

- a) 0.0
- b) 0.25
- c) 0.75
- d) 0.50
- e) 1.0

[□ Jump to solution](#)

Question 8

Compute the sigmoid value at $z = 2$.

- a) About 0.50
- b) About 0.69
- c) About 0.95
- d) About 0.88
- e) About 0.99

[□ Jump to solution](#)

Question 9

What can the sigmoid function NEVER output?

- a) A negative slope
- b) The value 0.5
- c) A value below 0 or above 1 — sigmoid is bounded strictly in $(0, 1)$ for every real input
- d) A symmetric S-shape
- e) A value of exactly 0.50

[□ Jump to solution](#)

Question 10

In a logistic regression model, what is the FULL one-sentence description of how a prediction is made?

- a) Compute the raw distance between the row and the nearest neighbor
- b) Compute the dot product and report it directly as the probability
- c) Run the row through a decision tree
- d) Compute the mean of the target column for the training set
- e) Compute a weighted sum of the features (plus an intercept), then squash that sum through the sigmoid function to obtain a probability between 0 and 1

[□ Jump to solution](#)

Question 11

In a Capital One credit model, the linear part of logistic regression evaluates to $z = 1.1$ for a given applicant. What is the applicant's predicted probability of approval (use $\text{sigmoid}(1.1) \approx 0.75$)?

- a) About 0.10
- b) About 0.25
- c) About 0.75
- d) About 0.50
- e) About 1.10

[□ Jump to solution](#)

Question 12

Which expression equals the **odds** of an event whose probability is p ?

- a) $p / (1 - p)$
- b) $p \times (1 - p)$
- c) $1 - p$
- d) $p + (1 - p)$
- e) $1 / p$

[□ Jump to solution](#)

Question 13

A model predicts probability 0.80 that a Spotify subscriber will renew. What are the corresponding odds?

- a) 4 to 1 (i.e., the renewal is four times as likely as the non-renewal)
- b) 1 to 4
- c) 0.80 to 1
- d) 8 to 10
- e) 1.25 to 1

[□ Jump to solution](#)

Question 14

What does **log-odds** (the logarithm of the odds) represent in a logistic regression model?

- a) Always a positive number bounded in $[0, 1]$
- b) The exact same thing as the odds
- c) The number of features
- d) The predicted probability rounded to two decimal places
- e) The linear combination of the features ($z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$); logistic regression is linear in log-odds even though it is non-linear in probability

[□ Jump to solution](#)

Question 15

Why do we work with **log-odds** instead of just probability when interpreting logistic regression?

- a) Probability cannot be computed in scikit-learn
- b) Log-odds always equals the predicted probability
- c) Log-odds is faster to compute

- d) Log-odds runs from $-\infty$ to $+\infty$ and is a linear function of the features, which makes coefficients interpretable as “a one-unit increase in x_j shifts the log-odds by β_j ” — a clean, additive statement that probabilities cannot match (probabilities are bounded and non-linear)
- e) Log-odds is required by Python

[□ Jump to solution](#)

Question 16

What is the relationship between a logistic-regression **coefficient** β and the **odds ratio** for that feature?

- a) The odds ratio equals β
- b) The odds ratio equals $1 / \beta$
- c) The odds ratio equals e^β — for a one-unit increase in the feature, the odds of the positive class are multiplied by e^β , holding other features constant
- d) The odds ratio equals β^2
- e) The odds ratio is unrelated to β

[□ Jump to solution](#)

Question 17

A Capital One default model has coefficient $+0.4$ on debt-to-income (per 0.1 increase). The odds ratio is $e^{0.4} \approx 1.49$. The BEST plain-English interpretation:

- a) Each 0.1 increase in DTI raises the predicted probability of default by exactly 49 percentage points
- b) Each 0.1 increase in DTI multiplies the predicted probability by 1.49
- c) Each 0.1 increase in DTI adds 1.49 to the predicted log-odds
- d) Each 0.1 increase in DTI multiplies the odds of default by about 1.49 (~50% higher odds), holding everything else constant
- e) DTI has no effect on default

[□ Jump to solution](#)

Question 18

A logistic-regression coefficient on `customer_satisfaction_score` is **-0.6**. Which statement is true?

- a) Higher satisfaction is associated with HIGHER predicted probability of churn
- b) Higher satisfaction is associated with LOWER predicted probability of churn (the negative coefficient means the log-odds — and therefore the probability — fall as the feature rises, holding other features constant)

- c) Higher satisfaction has no effect on churn
- d) The coefficient is invalid because it is negative
- e) The intercept is wrong

[□ Jump to solution](#)

Question 19

A logistic-regression coefficient on `is_premium_member` (a 0/1 indicator) is **+0.7**. The odds ratio is $e^{0.7} \approx 2.01$. Translate to a business audience:

- a) Premium members are 0.7 times as likely to convert
- b) The odds of conversion for premium members are about double those of non-members, holding other features constant
- c) Premium members convert with probability 0.70
- d) Premium members add 0.7 percentage points to conversion
- e) Membership has no measurable effect

[□ Jump to solution](#)

Question 20

Why is “an additional unit of x raises the **probability** of the outcome by β_j ” a WRONG interpretation of a logistic-regression coefficient?

- a) Because logistic regression has no coefficients
- b) Because probabilities are negative
- c) Because probability and odds are the same
- d) Because Python rounds differently
- e) Because probabilities are bounded in $[0, 1]$ and the sigmoid is non-linear: a one-unit change in x shifts the log-odds by β_j , but the corresponding change in probability depends on where on the S-curve you currently sit. Near the middle ($p \approx 0.5$) the probability moves a lot; near the tails ($p \approx 0.05$ or 0.95) it barely moves

[□ Jump to solution](#)

Question 21

How does logistic regression **learn** its coefficients?

- a) By minimizing the sum of squared errors, exactly like OLS
- b) By maximizing the likelihood of the observed labels (equivalently, minimizing the cross-entropy / log-loss). The optimization is iterative — there is no closed-form solution like OLS has
- c) By picking the coefficients that maximize accuracy on the training set
- d) By averaging the labels of the K nearest neighbors

- e) By tree splitting on each feature

[□ Jump to solution](#)

Question 22

Is there a closed-form formula for the optimal logistic-regression coefficients (analogous to OLS)?

- a) Yes — sklearn computes it directly
- b) Yes — sklearn uses the matrix $(X^T X)^{-1} X^T y$
- c) Yes — but only for two features
- d) Yes — the same OLS formula works
- e) No — the maximum-likelihood objective has no closed form, so sklearn solves it iteratively (gradient-based optimization or related methods); convergence depends on `max_iter`, the chosen `solver`, and feature scaling

[□ Jump to solution](#)

Question 23

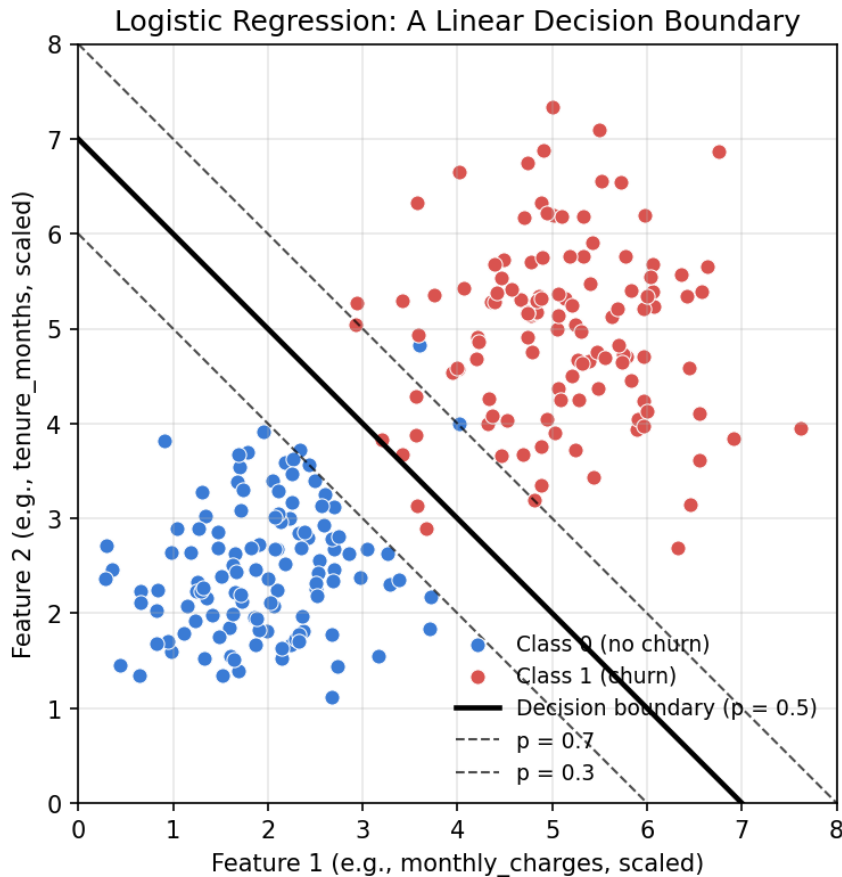
What is the **decision boundary** of a binary logistic regression in the original feature space?

- a) A circle
- b) A polynomial curve
- c) A linear hyperplane (a straight line in 2D, a flat plane in 3D), defined by the set of inputs where the predicted probability equals the chosen threshold (0.5 by default)
- d) A random shape selected each run
- e) A point

[□ Jump to solution](#)

Question 24

The plot below shows a Netflix churn model with two features (scaled), the decision boundary at $p = 0.5$, and the contours at $p = 0.3$ and $p = 0.7$.



Which statement is BEST supported by the plot?

- a) The decision boundary is a curve
- b) The model classifies every point as Class 0
- c) The two features are perfectly correlated
- d) The boundary at $p = 0.5$ is a straight line; nearby contours at $p = 0.3$ and $p = 0.7$ are parallel lines because the log-odds are linear in the features. Where the two classes overlap, the model gives intermediate probabilities (between 0.3 and 0.7)
- e) The model uses tree splits

[□ Jump to solution](#)

Question 25

What does the **default classification threshold** of 0.5 mean in scikit-learn's `LogisticRegression.predict`?

- a) Predict positive whenever the linear combination z exceeds 0.5
- b) Predict positive whenever fewer than 50% of features are missing
- c) Predict positive whenever the model has reached 50% accuracy
- d) Predict positive whenever the predicted probability of the positive class is ≥ 0.5 ; otherwise predict negative
- e) Threshold 0.5 is fixed and cannot be changed

[□ Jump to solution](#)

Question 26

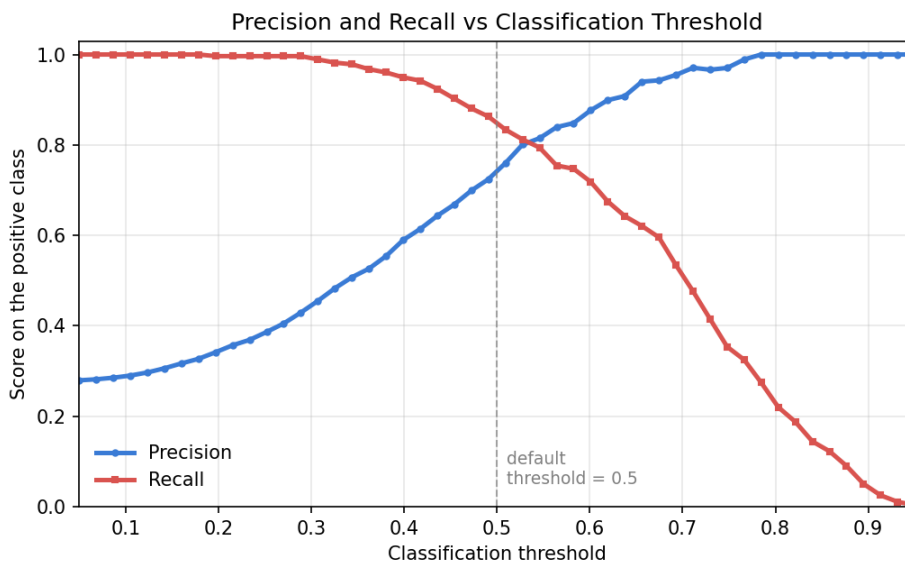
A Netflix retention team finds that retention emails are very cheap, but losing a subscriber is expensive. The default churn-classification threshold is 0.50. Which adjustment is most appropriate?

- a) Raise the threshold to 0.90 to flag fewer customers
- b) Lower the threshold (e.g., to 0.30) so more potential churners get flagged for a retention email — recall on the churn class rises, precision drops; with cheap emails and expensive churn, this is the right tradeoff
- c) Keep the threshold at 0.50 — it is always optimal
- d) Set the threshold to 1.0 to flag only certain churners
- e) Switch to a regression model

[□ Jump to solution](#)

Question 27

The plot below shows precision and recall on a fraud-detection model as the classification threshold is swept from 0.05 to 0.95.



Which statement is BEST supported by the plot?

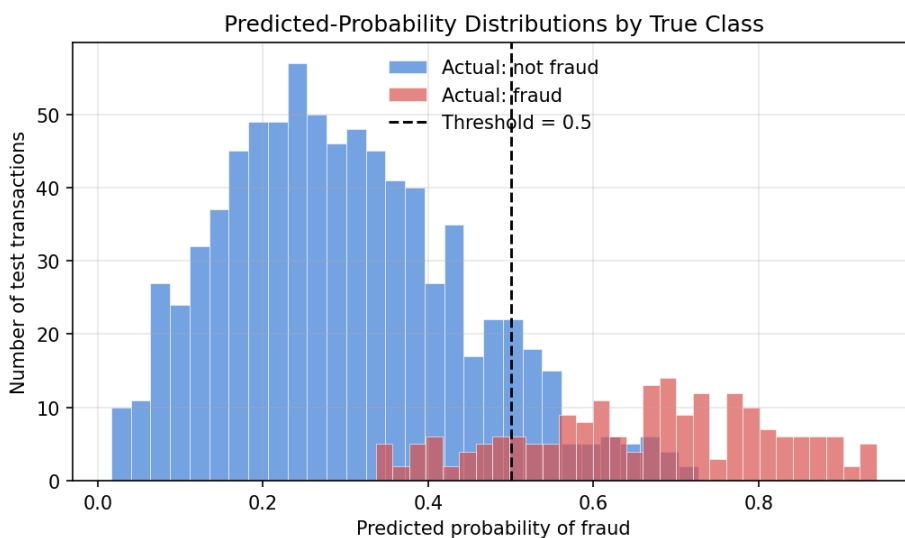
- a) Precision and recall always rise together
- b) The default 0.50 always maximizes both
- c) The two metrics are independent of the threshold

- d) Lowering the threshold increases recall (more positives are flagged) at the cost of precision (more of those flags are wrong); raising the threshold does the opposite. The team picks the threshold that matches the business cost asymmetry, not the default 0.50
- e) Recall is higher than precision at every threshold

[□ Jump to solution](#)

Question 28

The plot below shows the predicted-probability distributions on a fraud-detection test set, separated by the true class.



Which statement is BEST supported by the plot?

- a) The model assigns identical predicted probabilities to fraud and non-fraud
- b) The two distributions are visibly separated — non-fraud transactions concentrate at low predicted probabilities, fraud at high — but they overlap in the middle. The classification threshold determines how that overlap region is labeled, and lowering the threshold catches more fraud at the cost of more false alarms
- c) Every transaction is correctly classified
- d) The model is a random guess
- e) Fraud transactions all sit above 0.95

[□ Jump to solution](#)

Question 29

A logistic-regression fraud model is trained on a dataset with 99% “not fraud” and 1% “fraud”. With the default threshold of 0.5, the model predicts “not fraud” for almost every transaction. What is happening?

- a) The model has a bug
- b) The class imbalance means the optimization can achieve very low log-loss by mostly predicting the majority class. Combine threshold tuning (lower the cutoff) with `class_weight="balanced"` (which up-weights minority-class errors during fitting) and evaluate with precision/recall on the fraud class — not accuracy
- c) Logistic regression cannot handle binary problems
- d) The features were not scaled
- e) The intercept needs to be reset

[□ Jump to solution](#)

Question 30

What does `class_weight="balanced"` do in `LogisticRegression`?

- a) Adjusts the loss function to penalize errors on the minority class more heavily — implicitly equivalent to upsampling the minority class to match the majority. Useful baseline tool for imbalanced classification
- b) Adds extra rows to the minority class
- c) Drops rows from the majority class
- d) Changes the threshold from 0.5 to 0.9
- e) Always doubles the model's accuracy

[□ Jump to solution](#)

Question 31

In `LogisticRegression(C=0.01)` VS `LogisticRegression(C=100)`, which model has MORE regularization?

- a) `C=100` (larger numbers always mean more regularization)
- b) Neither applies regularization
- c) They are equally regularized
- d) `C=0.01` — sklearn's `C` is the INVERSE of regularization strength; smaller `C` means a stronger penalty on coefficient magnitude
- e) `C=0` (which is illegal in sklearn)

[□ Jump to solution](#)

Question 32

Which penalty setting in `LogisticRegression` performs **automatic feature selection** by driving some coefficients exactly to zero?

- a) `penalty="l1"` — analogous to Lasso, the L1 penalty produces sparse coefficient vectors and is the standard choice when you want a small, interpretable set of features

- b) `penalty="l2"`
- c) `penalty="none"`
- d) `penalty="elastic"` only
- e) `penalty="auto"`

[□ Jump to solution](#)

Question 33

Why is it important to **scale features BEFORE applying regularized logistic regression**?

- a) Logistic regression breaks without scaling
- b) Scaling is required for any model in scikit-learn
- c) The L1/L2 penalty acts on coefficient magnitudes. Without scaling, a feature in the millions (income) is naturally given a tiny coefficient, and a feature in $[0, 1]$ (a percentage) is given a huge coefficient — purely because of unit choice. The regularization penalty then lands unevenly across features. Scale (StandardScaler in a Pipeline) so the penalty applies fairly
- d) Scaling forces probabilities to sum to 1
- e) Scaling is purely cosmetic for logistic regression

[□ Jump to solution](#)

Question 34

How does logistic regression handle MORE than two classes (e.g., predicting Salesforce deal outcomes "Won", "Lost", or "Pending")?

- a) Two common approaches: One-vs-Rest (train one binary classifier per class; pick the class with the highest score), and Multinomial / Softmax (a single model that outputs a probability per class summing to 1). Multinomial is sklearn's default for multiclass
- b) It cannot handle multiclass problems
- c) It always uses K-Means under the hood
- d) It collapses the classes to two by random pairing
- e) It runs the binary version K times in series

[□ Jump to solution](#)

Question 35

What does `model.predict_proba(X_test)` return in a binary LogisticRegression?

- a) A single number — the test accuracy
- b) A confusion matrix

- c) An array of shape (n_test_rows, 2) where the second column is the predicted probability of the positive class and the first column is 1 minus that
- d) The training labels
- e) A pandas DataFrame with all training data

[□ Jump to solution](#)

Question 36

A team writes:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
proba = model.predict_proba(X_test)[: , 1]
```

What does y_pred contain vs proba?

- a) y_pred is the test set; proba is the train set
- b) y_pred is always the integer 1
- c) Both contain the same numbers
- d) y_pred is the model object; proba is the test labels
- e) y_pred contains 0 / 1 class labels (using the default threshold of 0.5 on proba); proba contains the predicted probability of the positive class for each test row, allowing the team to apply a custom threshold for cost-asymmetric decisions

[□ Jump to solution](#)

Question 37

Consider:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(C=1.0, max_iter=1000)),
])
pipe.fit(X_train, y_train)
```

Why is this pattern important?

- a) The pipeline ensures the scaler is fit on the training fold ONLY (no leakage during cross-validation), and the same fitted scaler is then applied to test data automatically. Regularized logistic regression in particular needs scaling, and the pipeline ensures the two steps are always coupled correctly
- b) Pipelines are required by sklearn
- c) Pipelines reduce model accuracy by ~5%
- d) Pipelines automatically tune c
- e) Pipelines avoid the need for `train_test_split`

[□ Jump to solution](#)

Question 38

Consider:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(C=0.1, penalty="l1", solver="liblinear")
model.fit(X_train_scaled, y_train)
print((model.coef_ == 0).sum(), "/", model.coef_.size, "coefficients zeroed")
```

What is this snippet doing?

- a) Counting the number of training rows
- b) Counting the number of test predictions
- c) Saving the model to disk
- d) Computing the test accuracy
- e) Fitting an L1-penalized logistic regression and printing how many coefficients the L1 penalty drove to exactly zero — Lasso-style automatic feature selection inside logistic regression

[□ Jump to solution](#)

Question 39

A logistic-regression model fails to converge and prints “ConvergenceWarning”. What is the FIRST thing you should check?

- a) Whether the labels are 0 / 1 (and not strings)
- b) The intercept is wrong
- c) The model is broken and should be replaced
- d) The training set is too large
- e) Three common causes, in priority order: (1) features were not scaled — bring in `StandardScaler` (in a Pipeline); (2) `max_iter` is too low — try `max_iter=1000` or higher; (3) two features are perfectly collinear — remove one. Switching `solver` (e.g., “liblinear” or “saga”) can also help on certain problems

[□ Jump to solution](#)

Question 40

A coefficient on a one-hot-encoded `region_East` (with `region_West` as the reference) is **+0.5**. Which statement is true?

- a) East customers' log-odds of the positive class are 0.5 higher than otherwise-identical West customers; equivalently, their odds are $e^{0.5} \approx 1.65\times$ higher
- b) East customers always default
- c) East customers are 50% of all customers
- d) `region_East` is a numeric feature that should be scaled
- e) The dummy was encoded incorrectly

[□ Jump to solution](#)

Question 41

A bank uses logistic regression to communicate to applicants: "Your probability of approval is 30%." For that statement to be honest, the model's predicted probabilities must be **calibrated** — when the model says 30%, roughly 30% of those applicants are in fact approved. Which scenario MOST requires calibration?

- a) Picking the single highest-risk customer for a phone call
- b) Quoting customers a numerical approval probability or expected interest rate, where the number on the screen has to match real-world frequencies
- c) Ranking customers by risk for marketing
- d) Building an ad-recommendation system
- e) Computing the AUC

[□ Jump to solution](#)

Question 42

Consider:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, model.predict(X_test)))
```

Why is this a better summary than `model.score(X_test, y_test)` for an IMBALANCED binary problem?

- a) `score()` is deprecated
- b) `classification_report` reports per-class precision, recall, F1, and the support count, exposing whether the model is actually catching the rare class — `score()` returns only overall accuracy, which can hide a model that always predicts the majority class
- c) `classification_report` computes the AUC
- d) `score()` is slower

e) `classification_report` requires no test labels

[□ Jump to solution](#)

Question 43

A team applies logistic regression with $C=1.0$ to a Capital One credit-default dataset and notes that two features (`monthly_income` and `annual_income / 12`) have wildly opposite coefficients. Most likely cause?

- a) Logistic regression cannot handle ratios
- b) The features were not standardized
- c) The intercept is wrong
- d) `class_weight="balanced"` was not set
- e) Multicollinearity. The two features are essentially the same column, so the optimization spreads coefficient mass across them in unstable ways. Drop one (or use stronger regularization, e.g., smaller C , or `penalty="l1"` to zero one out)

[□ Jump to solution](#)

Question 44

A logistic regression on a training set has an unusually large coefficient (e.g., $+12$) on a single feature. Which is the BEST first response?

- a) Suspect a problem and investigate: (i) is the feature highly correlated with the target by construction (target leakage)? (ii) was the feature unscaled, so its small numeric range produces a large coefficient? (iii) is the dataset tiny, so the optimizer is fitting noise? Apply scaling, regularization, and leakage checks before deploying
- b) Trust it and report it
- c) Add 12 more features
- d) Switch to KNN
- e) Set `random_state` to a different value

[□ Jump to solution](#)

Question 45

A logistic regression on a Lending Club dataset reports **train AUC = 0.95** and **test AUC = 0.65**. Diagnosis and remedy?

- a) The model is overfitting (large gap between train and test). Standard fixes: lower c (stronger regularization), reduce or drop weak / collinear features, scale features, or gather more training data
- b) The model is well-calibrated; deploy
- c) The model is underfit; add more features

- d) The intercept is wrong
- e) The labels are corrupt

[□ Jump to solution](#)

Question 46

A logistic regression with no `class_weight` and the default threshold reports **accuracy = 0.97** on a fraud dataset where 3% of transactions are fraud. Which conclusion is BEST?

- a) Excellent — deploy
- b) The model has converged
- c) Accuracy of 0.97 says almost nothing here. A trivial “predict not fraud” classifier already lands at 0.97. Look at the precision and recall on the fraud class: if recall is near zero, the model is not catching real fraud despite the apparent high accuracy
- d) Logistic regression cannot reach 0.97
- e) The intercept is too low

[□ Jump to solution](#)

Question 47

A logistic regression coefficient table reports:

feature	coef	odds ratio (e^{coef})
tenure_months	−0.05	0.95
monthly_charges	+0.012	1.012
contract_two_year	−1.30	0.27

A two-year contract customer (vs the reference category of month-to-month), holding everything else equal, has odds of churn equal to:

- a) 1.30 times higher
- b) 0.27 times the odds of an otherwise-identical month-to-month customer (about 73% lower) — strong protective effect
- c) Exactly 0.5
- d) Equal to the month-to-month odds
- e) The negative coefficient invalidates the row

[□ Jump to solution](#)

Question 48

In a multinomial logistic regression for Salesforce deal stage (Won / Lost / Pending), what does `predict_proba(X_test)` return for each row?

- a) A single integer
- b) The training labels
- c) A vector of K probabilities (one per class) that sum to 1; the class with the highest probability is what `predict()` returns by default
- d) The class with the lowest probability
- e) The Gini impurity

[□ Jump to solution](#)

Question 49

A junior analyst writes:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(C=1.0)
model.fit(X_train, y_train)          # X_train is raw, unscaled
print("Coef on income:", model.coef_[0][income_idx])
```

The coefficient on `income` is reported as 0.0000003, which the analyst interprets as “income has almost no effect on the outcome.” What is the BEST critique?

- a) The coefficient is correct — income has no effect
- b) Logistic regression is the wrong model
- c) The coefficient looks tiny only because `income` is on a huge unit scale (dollars). To compare effect sizes across features, standardize all features first (StandardScaler in a Pipeline) and re-examine the coefficients on a per-standard-deviation basis. The small absolute number does NOT mean small effect
- d) The intercept is too small
- e) The feature should be removed

[□ Jump to solution](#)

Question 50

A logistic-regression model produces these top-3 features by absolute coefficient magnitude (after standardization):

feature	std. coef	direction
tenure_months	−1.20	longer tenure □ less likely to churn

feature	std. coef	direction
monthly_charges	+0.85	higher charges ☐ more likely to churn
contract_two_year	−1.05	two-year contract ☐ less likely to churn

If you communicate this to a non-technical product manager, which framing is BEST?

- a) "The model has high R^2 "
- b) "Customer tenure causes churn"
- c) "These are the three most influential drivers of churn in our model: the longer the tenure, the less likely a customer churns; higher monthly charges raise the churn risk; and customers on two-year contracts are far less likely to churn — likely because the contract itself locks them in"
- d) "We need more data before we can interpret coefficients"
- e) "Logistic regression does not produce feature importance"

[□ Jump to solution](#)

Question 51

A logistic-regression model is calibrated (predicted probabilities match observed frequencies). Suddenly you switch to a Random Forest of 500 trees. Without further work, the Random Forest's `predict_proba` outputs are:

- a) Always perfectly calibrated by construction
- b) Mathematically guaranteed to outperform logistic regression
- c) Identical to the logistic-regression outputs
- d) Only categorical
- e) Often POORLY calibrated — tree-ensembles tend to push probabilities toward 0 or 1 and need post-hoc calibration (e.g., `CalibratedClassifierCV` with sigmoid/Platt scaling or isotonic regression) when calibrated probabilities matter

[□ Jump to solution](#)

Question 52

A team trains logistic regression and Lasso logistic regression on a dataset with 200 features. Lasso retains 38 features (162 zeroed). Which statement about the kept features is BEST?

- a) The 38 retained features are guaranteed to be causally related to the target
- b) Lasso always retains exactly half the features
- c) The 38 features are guaranteed to also win on a different dataset
- d) The 162 zeroed features have zero predictive value in any context

- e) The 38 retained features are the ones the model found most useful at the chosen c value, given the standardized training data. With heavy multicollinearity, Lasso may have arbitrarily picked one of each correlated group; report the surviving set, but treat it as ONE plausible interpretable subset, not the unique truth

[□ Jump to solution](#)

Question 53

A logistic regression for credit approval at Capital One assigns probability 0.95 to one applicant and 0.05 to another. Which statement about the **decision** the bank makes is BEST?

- a) The bank automatically approves both
- b) The probabilities have no business meaning
- c) The bank always rejects both
- d) The model alone defines the bank's policy
- e) With the default 0.5 threshold the bank approves the 0.95 applicant and denies the 0.05 applicant. With a custom threshold (e.g., 0.30) the bank could approve more applicants — at the cost of taking on more default risk. The threshold encodes the bank's risk appetite, not a property of the model

[□ Jump to solution](#)

Question 54

Compared with linear regression, logistic regression's coefficients have a **multiplicative** business meaning rather than additive. Which is the most useful translation rule for a stakeholder?

- a) "A coefficient of β on a feature means a one-unit increase in that feature multiplies the ODDS of the positive class by e^β , holding everything else constant. Sign tells you direction; size of e^β tells you magnitude"
- b) "A coefficient of β means the OUTCOME is multiplied by β "
- c) "A coefficient of β means the predicted outcome rises by β units"
- d) "Coefficients have no interpretation in logistic regression"
- e) "The coefficient equals the predicted probability"

[□ Jump to solution](#)

Question 55

A team has a logistic-regression credit-default model. Train AUC = 0.81, test AUC = 0.79. They want a small, interpretable feature set for regulators. Final recipe?

- a) Keep all features; report only AUC
- b) Drop the test set

- c) Switch to a deep neural network for explainability
- d) Standardize features in a Pipeline; fit Lasso (`penalty="l1"`) logistic regression with cross-validated `c`; inspect the surviving coefficients (sign and odds ratio); choose a threshold that matches the cost asymmetry of false positives vs false negatives; report the model card with this information
- e) Use only `class_weight="balanced"` and skip the Pipeline

[☐ *Jump to solution*](#)

Part 2: Answers and Explanations

Answer 1

Correct: (c). Logistic regression is purpose-built for binary classification. It outputs a probability between 0 and 1 (via the sigmoid), can be turned into yes/no with a chosen threshold, and produces interpretable coefficients — the right starting point for credit-card approval, churn, fraud, conversion, and many other binary problems.

[☐ Back to question](#)

Answer 2

Correct: (b). Linear regression treats binary 0/1 targets as continuous, so its predictions can drift outside $[0, 1]$ (negative or above 1) — nonsensical as probabilities. Its squared-error loss does not match a binary outcome's distribution, and a unit increase in x is given the same effect everywhere on the 0–1 scale, which is also wrong. Logistic regression fixes all of these issues.

[☐ Back to question](#)

Answer 3

Correct: (d). Logistic regression predicts the probability of the positive class. A separate threshold then turns that probability into a yes/no decision. The threshold is a business choice, not a model property.

[☐ Back to question](#)

Answer 4

Correct: (a). Logistic regression IS a linear classifier. Its decision boundary in the feature space is a straight line / flat plane. The non-linearity (the S-shaped sigmoid) sits only in the final squashing step that converts the linear combination into a probability — the boundary itself is linear.

[☐ Back to question](#)

Answer 5

Correct: (b). Logistic regression wins when transparency, calibrated probabilities, named-feature interpretability, regulatory defensibility, or a fast baseline matters. Random Forest typically beats logistic regression on raw accuracy when the relationship is non-linear — but at the cost of these properties.

[☐ Back to question](#)

Answer 6

Correct: (d). Sigmoid (logistic) takes any real number into (0, 1) with a smooth S-shape. $z = 0$ maps to 0.5; large positive z approaches 1; large negative z approaches 0. This shape is exactly what a probability needs.

[□ Back to question](#)

Answer 7

Correct: (d) 0.50. $\text{sigmoid}(0) = 1 / (1 + e^0) = 1 / (1 + 1) = 1 / 2 = 0.50$. The plot's marked dot at ($z=0$, $p=0.5$) confirms it.

[□ Back to question](#)

Answer 8

Correct: (d) About 0.88. $\text{sigmoid}(2) = 1 / (1 + e^{-2}) = 1 / (1 + 0.135) \approx 1 / 1.135 \approx 0.881$.

[□ Back to question](#)

Answer 9

Correct: (c). $\text{sigmoid}(z)$ is bounded strictly between 0 and 1 for any real input. It approaches 0 and 1 asymptotically but never reaches either, and 0.5 is achieved only at $z = 0$.

[□ Back to question](#)

Answer 10

Correct: (e). A logistic regression prediction is two steps: a linear combination $z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$, then a sigmoid squashing of z into $[0, 1]$. That probability becomes a class label by comparing to the chosen threshold.

[□ Back to question](#)

Answer 11

Correct: (c) About 0.75. $\text{sigmoid}(1.1) = 1 / (1 + e^{-1.1}) \approx 1 / (1 + 0.333) \approx 0.750$. The applicant has a 75% predicted probability of approval — well above the default 0.5 threshold, so the model would predict “approve.”

[□ Back to question](#)

Answer 12

Correct: (a) $p / (1 - p)$. Odds is the ratio of the probability of the event to the probability of its complement. A 0.80 probability gives odds $0.80 / 0.20 = 4$ ("4 to 1").

[□ Back to question](#)

Answer 13

Correct: (a) 4 to 1. Odds = $p / (1 - p) = 0.80 / 0.20 = 4$. Renewal is four times as likely as non-renewal under this prediction.

[□ Back to question](#)

Answer 14

Correct: (e). Log-odds is the linear part of the logistic regression model: $z = \beta_0 + \sum_j \beta_j x_j$. Logistic regression is linear in log-odds even though it is non-linear in probability. This is why coefficients have clean interpretations on the log-odds scale.

[□ Back to question](#)

Answer 15

Correct: (d). Log-odds runs over the full real line and is a linear function of the features, so each coefficient has a simple, additive meaning ("a one-unit increase in x_j shifts the log-odds by β_j "). Probabilities are bounded and non-linear in the features, which makes equivalent statements much messier.

[□ Back to question](#)

Answer 16

Correct: (c). The odds ratio for feature j is e^{β_j} . A one-unit increase in x_j multiplies the odds of the positive class by e^{β_j} , holding the other features constant. This multiplicative interpretation is the standard way to describe logistic-regression coefficients to a business audience.

[□ Back to question](#)

Answer 17

Correct: (d). Each 0.1 increase in DTI multiplies the odds of default by about 1.49 — roughly 50% higher odds per 0.1 DTI step. Note: odds, not probability. The probability move depends on where on the S-curve you start.

[□ Back to question](#)

Answer 18

Correct: (b). A negative coefficient means the log-odds decrease as the feature rises, holding other features constant. Higher satisfaction lowers the predicted probability of churn — exactly the direction one would expect.

[□ Back to question](#)

Answer 19

Correct: (b). $e^{0.7} \approx 2.01$, so the odds of conversion for a premium member are about double those of an otherwise-identical non-member. Note the odds language — not probability. If a non-member's conversion probability is 0.10 (odds 0.111), a premium member has odds ≈ 0.223 , probability ≈ 0.18 . Near 0.5 the probability change would be much larger.

[□ Back to question](#)

Answer 20

Correct: (e). Probabilities are bounded and the sigmoid is S-shaped. A coefficient maps cleanly to a CONSTANT change in log-odds (or a constant multiplicative change in odds), but the corresponding probability change depends on where you are on the S-curve. This is the single most common interpretation error in business audiences — be precise about odds vs probability.

[□ Back to question](#)

Answer 21

Correct: (b). Logistic regression maximizes the likelihood of the observed labels (equivalently, minimizes the cross-entropy / log-loss). Unlike OLS, this objective has no closed-form solution, so sklearn solves it iteratively. Convergence depends on `max_iter`, the chosen `solver`, and feature scaling.

[□ Back to question](#)

Answer 22

Correct: (e). Maximum-likelihood logistic regression has no closed form. `sklearn` solves it iteratively. If you see “ConvergenceWarning,” scaling the features, raising `max_iter`, or switching `solver` usually fixes it.

[□ Back to question](#)

Answer 23

Correct: (c). The decision boundary is the set of inputs where the predicted probability equals the threshold (default 0.5). At the default threshold this corresponds to $z = 0$ — a linear hyperplane in the original feature space. Curved decision boundaries appear only when you add non-linear features (interactions, polynomials, kernel transforms) before fitting.

[□ Back to question](#)

Answer 24

Correct: (d). The boundary at $p = 0.5$ is the line where the linear combination z equals 0. The contour at $p = 0.7$ is where z equals a positive constant; $p = 0.3$ where z equals a negative constant. Both are parallel to the boundary because z is linear in the features. The overlap region between the classes is where the model is least confident — exactly where the threshold choice matters most.

[□ Back to question](#)

Answer 25

Correct: (d). `LogisticRegression.predict(...)` thresholds `predict_proba(...)`’s probability of the positive class at 0.5 by default. It is a convenience — for cost-asymmetric problems you should usually do the threshold yourself: `(model.predict_proba(X_test)[: , 1] >= custom_t).astype(int)`.

[□ Back to question](#)

Answer 26

Correct: (b). When false negatives (a missed churner) are far costlier than false positives (an unnecessary email), lower the threshold to flag more potential churners. Recall on the churn class rises, precision drops — and the dollar tradeoff is favorable when the intervention is cheap.

[□ Back to question](#)

Answer 27

Correct: (d). As the threshold falls, more cases get predicted positive — recall rises, precision falls. As the threshold rises, the opposite. The right threshold depends on the relative cost of false negatives vs false positives. The default 0.5 is a starting point, not the answer for every business problem.

[□ Back to question](#)

Answer 28

Correct: (b). When the predicted-probability distributions are well-separated (two distinct humps), the model has learned to discriminate the classes — but real models always have some overlap in the middle. Where you draw the threshold determines how the overlap region is labeled. A skilled team plots this distribution before choosing a threshold.

[□ Back to question](#)

Answer 29

Correct: (b). With 99/1 class imbalance, the optimization minimizes log-loss most easily by mostly predicting the majority class. Combine `class_weight="balanced"` (up-weights minority errors during training), threshold tuning (lower the cutoff), and per-class evaluation (precision/recall/F1, not accuracy). Those three together usually solve the problem.

[□ Back to question](#)

Answer 30

Correct: (a). `class_weight="balanced"` weights each class inversely proportional to its frequency. Minority-class errors then count more in the loss, so the optimizer pays attention to them. It is a clean, leakage-free alternative to manual oversampling for many problems.

[□ Back to question](#)

Answer 31

Correct: (d) C = 0.01. sklearn's `c` is the INVERSE of regularization strength. Smaller `C` = stronger penalty on coefficient magnitude. (This convention differs from Ridge/Lasso `alpha` and trips up beginners — a useful mnemonic: "C for Confidence in the data — small C means little confidence, regularize harder.")

[□ Back to question](#)

Answer 32

Correct: (a) `penalty="l1"`. L1 penalty produces sparse coefficients — many become exactly zero. This is the right choice when you want a small, interpretable feature subset (analogous to Lasso for linear regression). Note that you typically need `solver="liblinear"` or `solver="saga"` with `penalty="l1"`.

[☐ Back to question](#)

Answer 33

Correct: (c). L1 / L2 penalties act on coefficient magnitudes; without scaling, the magnitudes reflect both feature importance AND the units of measurement, so the penalty applies unevenly across features purely because of scale. Standardize (StandardScaler in a Pipeline) so the penalty applies fairly. Plain unregularized logistic regression does not need scaling for accuracy.

[☐ Back to question](#)

Answer 34

Correct: (a). One-vs-Rest trains K binary classifiers (one per class); Multinomial / Softmax trains a single model with K coefficient vectors so the per-class probabilities sum to 1. sklearn's default for multiclass is multinomial. Both are usable for K = 3, 4, ..., dozens.

[☐ Back to question](#)

Answer 35

Correct: (c). `predict_proba(X_test)` returns shape `(n_test, 2)` for binary problems: column 0 is P(class 0), column 1 is P(class 1). For multiclass, the shape is `(n_test, n_classes)`. Slicing `[:, 1]` gives the positive-class probability commonly used for thresholding.

[☐ Back to question](#)

Answer 36

Correct: (e). `predict()` returns the integer class labels using the default 0.5 threshold on `predict_proba()`. `predict_proba()` returns the underlying probabilities — the team can apply a custom threshold for a cost-asymmetric decision (lower threshold for higher recall, higher threshold for higher precision).

[☐ Back to question](#)

Answer 37

Correct: (a). The pipeline ensures the scaler is fit on each training fold (no leakage during cross-validation) and the same fitted scaler is applied to the held-out fold and to test data automatically. Regularized logistic regression in particular needs scaling, and the pipeline guarantees the two steps stay coupled.

[□ Back to question](#)

Answer 38

Correct: (e). L1-penalized logistic regression drives some coefficients exactly to zero. Counting zeros tells you how many features the L1 penalty effectively dropped — Lasso-style automatic feature selection inside logistic regression. Adjust `c` to control how aggressive the pruning is.

[□ Back to question](#)

Answer 39

Correct: (e). `ConvergenceWarning` is almost always one of three things: (1) features on wildly different scales — fix with `StandardScaler` in a Pipeline; (2) `max_iter` too low — bump to 1000+; (3) perfect or near-perfect collinearity — drop a feature. Switching `solver` to “liblinear” or “saga” sometimes resolves edge cases.

[□ Back to question](#)

Answer 40

Correct: (a). With one-hot encoding (West dropped as the reference), each remaining dummy’s coefficient is the log-odds difference vs the reference. A +0.5 coefficient on `region_East` means East customers’ log-odds are 0.5 higher than West customers, corresponding to odds about $e^{0.5} \approx 1.65\times$ higher.

[□ Back to question](#)

Answer 41

Correct: (b). Calibration matters when the predicted probability is consumed directly (quoting risk, expected interest rate, expected value with a cost matrix). For pure ranking tasks (top-N marketing, ad ranking), miscalibrated but well-ordered scores are fine. Logistic regression tends to be reasonably well-calibrated; tree ensembles and SVMs often need post-hoc calibration.

[□ Back to question](#)

Answer 42

Correct: (b). `classification_report` shows per-class precision, recall, F1, and support — exactly what you need to see if the model is catching the rare class on imbalanced problems. `score()` returns overall accuracy, which can hide a “predict majority class always” model.

[□ Back to question](#)

Answer 43

Correct: (e). Multicollinearity. The two features are essentially the same column on different scales, so the optimizer spreads coefficient mass across them in unstable, often opposite-signed ways. Drop one, raise regularization (smaller `c`), or move to L1 to zero one out automatically.

[□ Back to question](#)

Answer 44

Correct: (a). Large coefficients in logistic regression are a yellow flag. The most common causes: target leakage (a feature is a deterministic function of the label), unscaled features (a feature on a small numeric range produces a big coefficient), or tiny dataset (the optimizer is fitting noise). Investigate before deploying.

[□ Back to question](#)

Answer 45

Correct: (a). A 30-point gap between train AUC (0.95) and test AUC (0.65) is the textbook signature of overfitting. Standard fixes for logistic regression: lower `c` (stronger regularization), reduce or drop weak / collinear features, scale features in a Pipeline, or gather more training data.

[□ Back to question](#)

Answer 46

Correct: (c). With 3% positive class, “predict not fraud” alone scores 0.97 accuracy and catches zero fraud. Always pair accuracy with the per-class precision and recall on imbalanced problems — and consider `class_weight="balanced"` and threshold tuning.

[□ Back to question](#)

Answer 47

Correct: (b). The odds ratio of 0.27 means a two-year contract customer's odds of churn are about 27% of an otherwise-identical month-to-month customer's odds — roughly a 73% reduction. This is a strong protective effect and a classic finding in telecom churn data.

[□ Back to question](#)

Answer 48

Correct: (c). For multiclass, `predict_proba()` returns one probability per class per row, with each row summing to 1. `predict()` returns the class with the highest probability per row.

[□ Back to question](#)

Answer 49

Correct: (c). A coefficient is on the log-odds scale per UNIT change in the feature. Income is in dollars, so a one-dollar change in income produces a tiny log-odds shift — but that does not mean income is unimportant. Standardize all features (StandardScaler in a Pipeline) and re-examine the coefficients on a per-standard-deviation basis to compare effect sizes apples-to-apples.

[□ Back to question](#)

Answer 50

Correct: (c). This framing ties each standardized coefficient to the business meaning, names the direction, and adds a domain-plausible explanation (contract lock-in). It is the kind of summary a non-technical PM can act on.

[□ Back to question](#)

Answer 51

Correct: (e). Tree-based ensembles (Random Forest, gradient boosting) often produce probability outputs that lean too aggressively toward 0 or 1. If the probability has to mean something (risk quoting, expected-value decisions), wrap the model in `CalibratedClassifierCV(method="sigmoid")` or `method="isotonic"` to recalibrate the outputs.

[□ Back to question](#)

Answer 52

Correct: (e). Lasso's surviving features are the ones that most consistently contribute at the chosen c on the standardized training data. With multicollinearity, the choice of which feature in a correlated group survives can be somewhat arbitrary. Report the surviving subset, but understand it is one plausible interpretable model — not the unique truth.

[□ Back to question](#)

Answer 53

Correct: (e). The model produces probabilities; the threshold turns probabilities into approve/deny decisions. Different thresholds match different appetites for risk vs market share. The threshold is a business policy choice, not a property of the model.

[□ Back to question](#)

Answer 54

Correct: (a). Logistic-regression coefficients move ODDS multiplicatively. The clean translation rule: "Each one-unit increase in x_j multiplies the odds of the positive class by e^{β_j} , holding everything else constant." Sign tells direction; e^{β} tells magnitude. Resist the temptation to translate β as a probability change unless you also state where on the S-curve you are.

[□ Back to question](#)

Answer 55

Correct: (d). This is the standard production recipe: standardize in a Pipeline, use L1 + cross-validated c for a sparse, interpretable feature set, choose a threshold matched to the cost asymmetry, and document everything in a model card. The (train, test) AUC gap is small (0.81 vs 0.79), so the model is not overfitting and is ready to deploy with the documented threshold.

[□ Back to question](#)